**Project 1: MapReduce, Word Count Across Multiple Files**

**Problem Statement**

**Students will implement a MapReduce program in C using MPI to count the frequency of each word in a collection of text files. The program will:**

1. **Map Step: Process each file (or chunk of a file) in parallel, outputting intermediate key-value pairs (word, 1).**
2. **Shuffle/Sort Step: Group the same words together across all processes.**
3. **Reduce Step: Sum the counts for each word to get the total frequency.**

**This project demonstrates how MapReduce simplifies distributed data processing for real-world applications, like text analytics or log analysis.**

---

**Objectives**

1. **Implement the MapReduce workflow (Map, Shuffle, Reduce) using MPI.**
2. **Process multiple text files in parallel efficiently.**
3. **Count the frequency of each unique word across all files.**
4. **Combine results to produce a final word count table.**
5. **Analyze performance and speedup for different numbers of files and processes.**

Sample Input

- `file1.txt`

```nginx
hello world
hello mapreduce
```

- `file2.txt`

```kotlin
mapreduce is fun
hello world
```

Expected Output

```kotlin
hello: 3
world: 2
mapreduce: 2
is: 1
fun: 1
```